

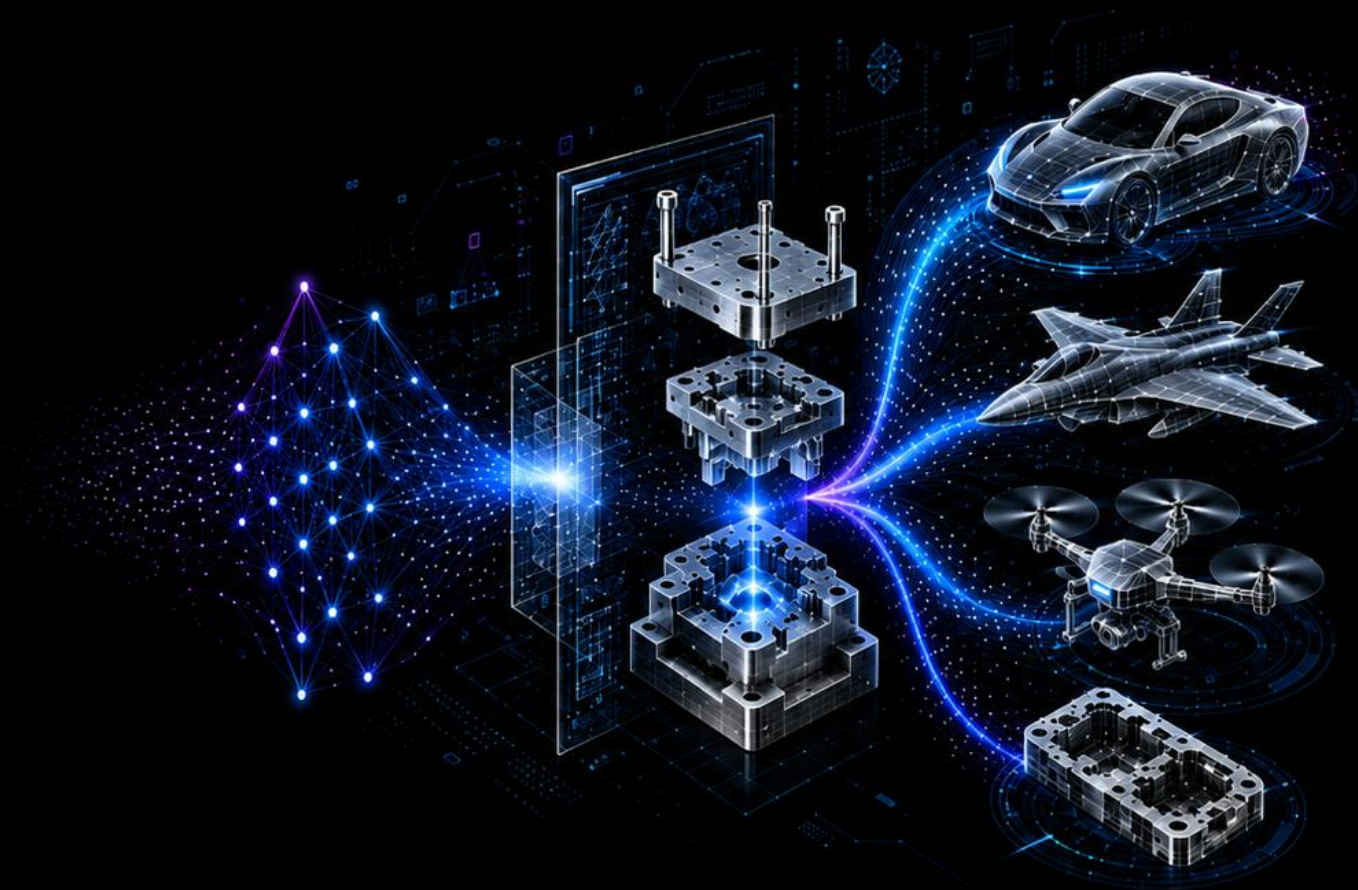
정규 스터디

7장 (시계열 분석)

최준석

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과



Intelligent Design
for Engineering
Applications Lab

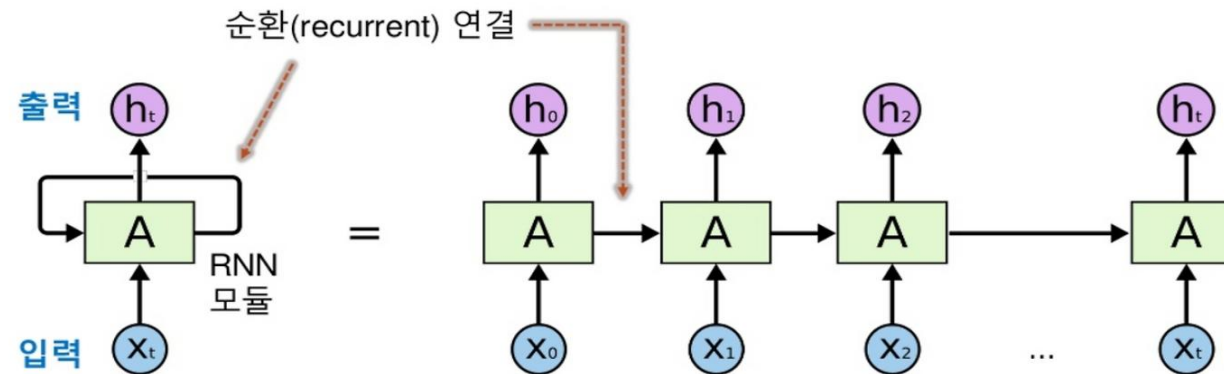
RNN의 개념 설명

✓ RNN이란?

: RNN의 R은 Recurrent(반복되는)로, 시간적으로 연속성이 있는 데이터를 처리하기 위한 모델.
즉, 이전 은닉층이 현재 은닉층의 입력이 된다.

✓ 다른 네트워크와의 차이점

: 현재까지의 입력데이터를 요약하여 “기억”한다.



RNN의 입력과 출력 유형

✓ 일대일

⇒ 순방향에 가까움

✓ 일대다

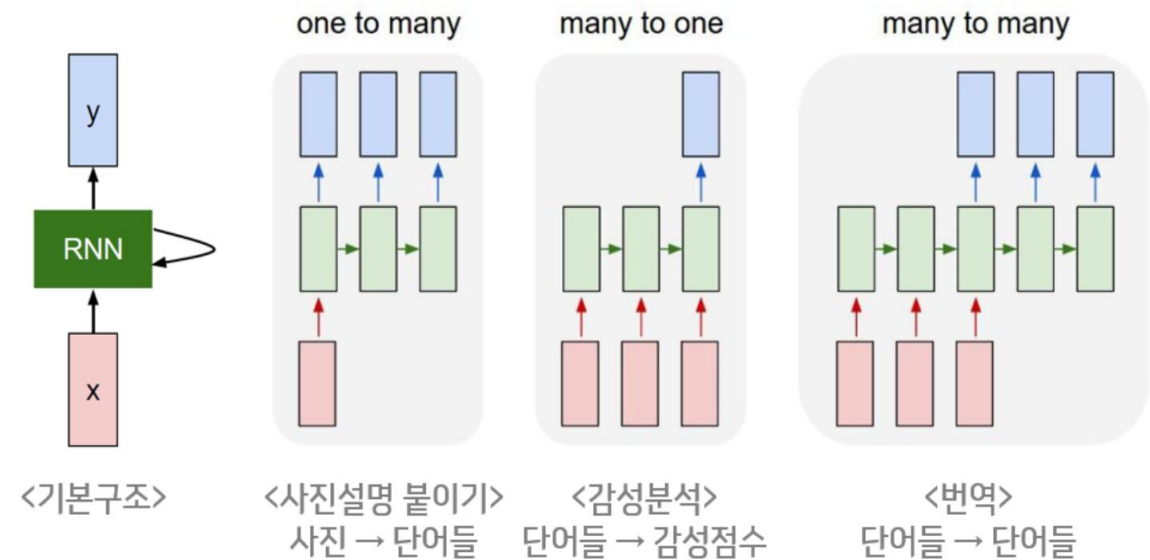
⇒ 이미지를 입력해 이미지에 대한 설명을 문장을 출력하는 예

✓ 다대일(중요)

⇒ 문장을 입력해서 긍정 / 부정 / 중립 을 출력하는 감성 분석기

✓ 다대다

⇒ 언어를 번역하는 자동 번역기의 예



■ RNN의 수식

✓ 은닉층에서의 수식

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h)$$

$$\hat{y}_t = W_y h_t + b_y$$

- x_t : t 시점의 입력
- h_{t-1} : 이전 시점 까지의 정보를 담은 은닉 상태
- h_t : 현재 시점까지 반영한 새로운 은닉 상태
- $\hat{y}_t$: 예측 값
- W_x, W_h, W_y, b : 학습하는 파라미터

✓ 출력층에서의 수식

$$P(y | x) = \text{softmax}(W_o h_T + b_o)$$

예) $P(\text{긍정}) = 0.85, \quad P(\text{부정}) = 0.15$

⇒ 위와 같이 출력층에서는 softmax 함수를 사용하여,
문장을 긍정 또는 부정으로 분류

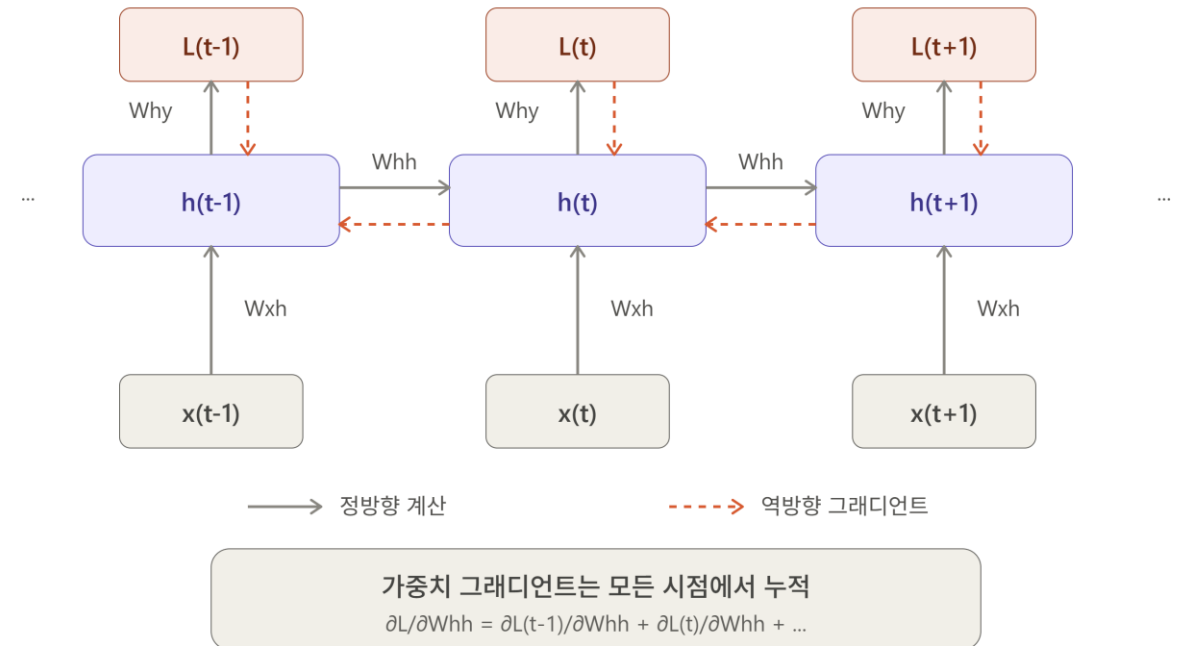
RNN 역전파와 한계

✓ RNN의 역전파 한계

: BPTT(RNN의 역전파)는 오차가 멀리 전파될 때, 계산량이 많아지고 전파되는 양이 점차 적어지는 (기울기 소멸 문제) 문제가 발생한다.

✓ 보완점

: 오차를 현재단계에서 특정 단계(보통 5단계 이전)까지만 전파 시키는 “생략된 - BPTT”를 사용하고, 대체로는 LSTM 또는 GRU를 많이 사용.



RNN의 전체 코드 흐름

Part 1
Introduction

Part 2
Method

Part 3
Results &
Discussion

Part 4
Conclusion

- ✓ Dataset: 데이터 입력
- ✓ Tokenizer: 토큰화
- ✓ Vocabulary: 10회 미만으로 사용된 단어는 사용 X
- ✓ 단어 → 정수 번호: (정수 인코딩) 각 토큰을 어휘 사전의 번호로 변환
- ✓ Padding + DataLoader: 패딩을 통한 배치 길이 맞춤
- ✓ Embedding: 정수 번호의 의미를 가진 실수 벡터로 변환
- ✓ RNN: 임베딩된 단어 벡터를 RNN에 입력
- ✓ 마지막 hidden state: 문장 전체 분류 시 마지막 은닉 상태 활용
- ✓ Linear layer: 마지막 은닉 상태를 통해 긍정·부정 점수로 변환
- ✓ Loss 계산: 이진 감성분석을 통한 손실함수 계산
- ✓ BPTT: 손실을 통한 역전파 과정
- ✓ 모수 업데이트: 파라미터 업데이트 진행



```
Dataset
↓
Tokenizer
↓
Vocabulary(min_freq=10)
↓
단어 → 정수 번호
↓
Padding + DataLoader
↓
Embedding
↓
RNN
↓
마지막 hidden state
↓
Linear layer
↓
Loss 계산
↓
BPTT
↓
모수 업데이트
```

LSTM 과 기본 RNN 의 차이

차이점

- ✓ LSTM은 기본 RNN보다 계산량과 모수(파라미터) 수가 많다.
- ✓ LSTM의 역전파에서는 최종 오차가 모든 노드에 전파된다. 즉, 셀을 통해 중단 없이 전파

항목	기본 RNN	LSTM
수식	$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
상태	h_t 하나	h_t, C_t 두 개
역전파 핵심 경로	은닉 상태	셀 상태와 은닉 상태
반복되는 주요 항	W_h 와 $\tanh$	망각 게이트 f_t
상태 업데이트	비선형 변환	곱셈과 덧셈
장기 기울기	쉽게 소멸	비교적 잘 보존
게이트	없음	망각·입력·출력 게이트
계산량	적음	많음
장기 의존성	취약	상대적으로 강함

Part 1
Introduction

Part 2
Method

Part 3
Results &
Discussion

Part 4
Conclusion

■ LSTM이란?: RNN + 망각 게이트, 입력 게이트, 출력 게이트 형태의 구조

입력 게이트

- ✓ 의미: 현재 정보를 기억하기 위해서 만들어짐
- ✓ 형태: 과거 정보와 현재 데이터를 입력 받아 시그모이드(sigmoid)와 하이퍼볼릭 탄젠트를 기반으로 현재 정보에 대한 보존량을 결정

- ✓ 수식

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

- x_t : 현재 입력
- h_{t-1} : 직전 은닉 상태
- W_i, b_i : 학습하는 모수
- σ : 시그모이드 함수

망각 게이트

- ✓ 의미: 과거 정보를 어느 정도 기억할지를 결정
- ✓ 형태: 과거 정보와 현재 데이터를 입력 받아 시그모이드(sigmoid)를 취한 뒤 그 값을 과거 정보에 곱해줌

- ✓ 수식

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

- x_t : 현재 입력
- h_{t-1} : 직전 은닉 상태
- W_f, b_f : 학습하는 모수
- σ : 시그모이드 함수
- f_t : 망각 게이트의 출력

+ 시그모이드의 출력 범위는 0~1

후보 셀 & 셀 상태 업데이트

의미와 형태

- ✓ 후보 셀: $\tilde{C}_t$ 는 현재 시점에서 새롭게 기억할 내용의 후보
- ✓ 후보 셀의 수식

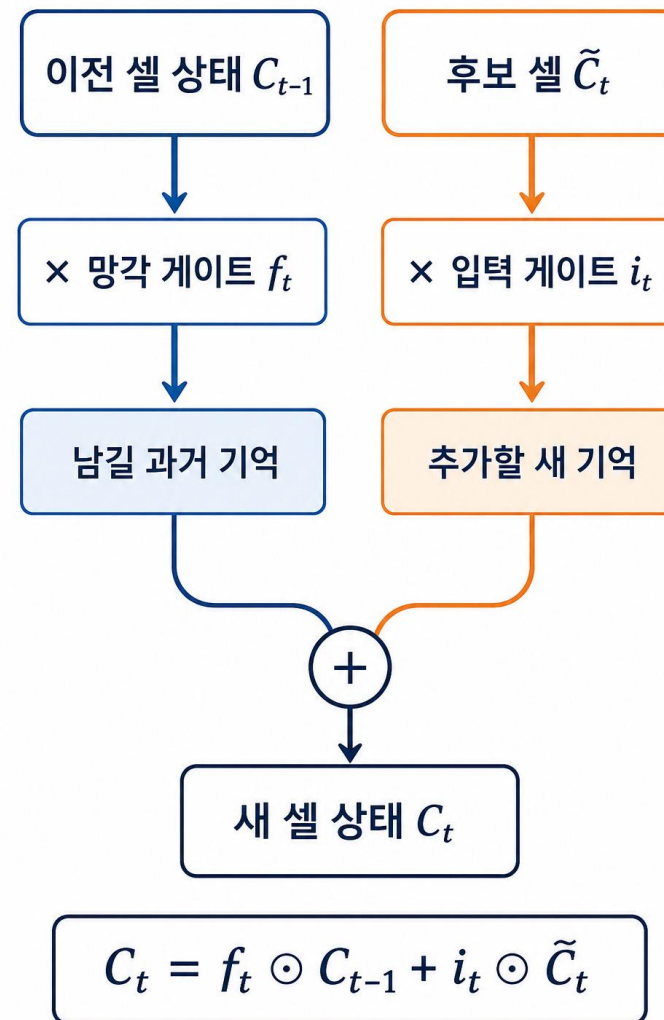
$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

- ✓ 셀 상태 업데이트
: 이전 기억에서 불필요한 부분은 잊고, 새로운 후보 기억을 추가
- ✓ 업데이트 수식

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

[$\odot$ 의 의미는 "점 단위 연산". 즉, 각각의 차원에 맞게 곱하거나 더하는 부호]

- i_t : 입력 게이트 [$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$]
- $\tilde{C}_t$: 새롭게 만들어진 후보 기억 [$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$]
- $i_t \odot \tilde{C}_t$: 새 기억 중 추가할 부분



■ 최종 상태 + 출력 게이트

✓ 출력 게이트: 업데이트된 셀 상태 C_t 중에서 현재 은닉 상태 h_t 로 얼마나 내보낼지 결정

✓ 출력 게이트 수식

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$$

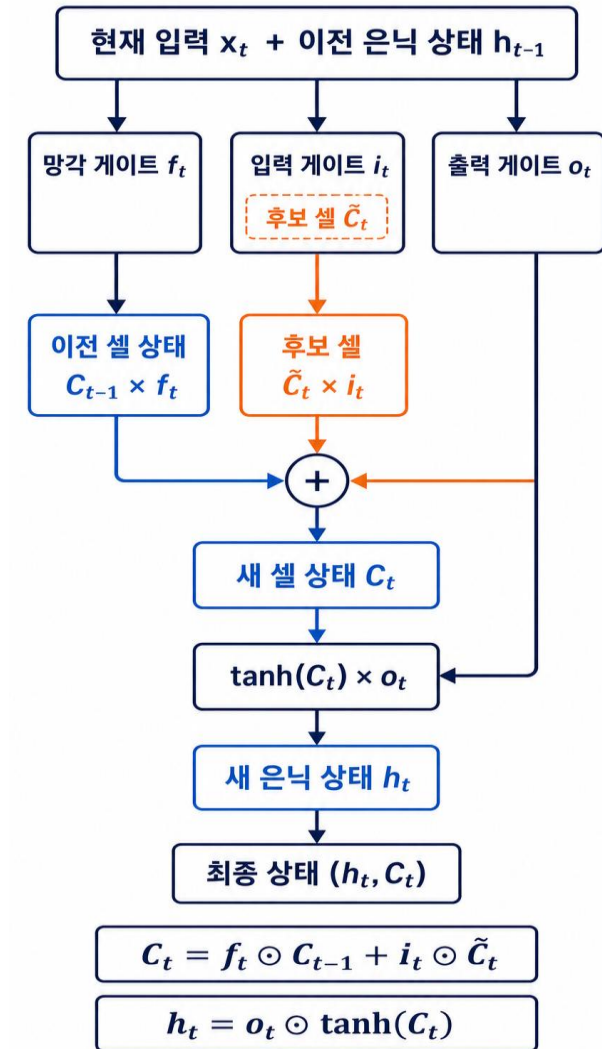
✓ 최종 상태: 업데이트 된 기억과 출력 게이트 값이 합쳐진 상태

✓ 최종 상태 수식

$$h_t = o_t \odot \tanh(C_t)$$

⇒ 마지막 분류·예측에서 h_T 를 사용한다.

최종 LSTM 구조



■ 추가적인 학습 내용

SGD

- ✓ LSTM 셀 구현에서 옵티마이저를 우리가 흔히 사용한 Adam이 아닌 SGD를 사용

✓ SGD란?

: 확률적 경사하강법(Stochastic Gradient Descent)의 약자로, 무작위로 선택한 데이터 하나를 사용해 신경망의 손실을 줄이는 방향으로 모수를 업데이트하는 최적화 방법

✓ 기본 원리

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

- $\nabla_{\theta} L$: 손실이 가장 빠르게 증가하는 방향
- 마이너스 기호: 손실이 감소하는 반대 방향으로 이동
- η : 학습률, 한 번에 이동하는 크기

✓ 왜 SGD를 사용하였는가?

⇒ 셀 예제에서는 학습 원리를 단순하게 보여주려고 SGD를 쓰고, 계층 예제에서는 성능과 학습 편의성을 위해 Adam을 쓴 것

옵티마이저와 손실 함수 지정 (셀 구현)

```
input_dim = 28
hidden_dim = 128
layer_dim = 1
output_dim = 10

model = LSTMModel(input_dim, hidden_dim, layer_dim, output_dim)
if torch.cuda.is_available():
    model.cuda()

criterion = nn.CrossEntropyLoss()
learning_rate = 0.1
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```



옵티마이저와 손실 함수 지정 (계층 구현)

```
num_epochs = 1000
learning_rate = 0.0001

input_size = 5
hidden_size = 2
num_layers = 1

num_classes = 1
model = LSTM(num_classes, input_size, hidden_size, num_layers, X_train_tensors_f.shape[1])

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

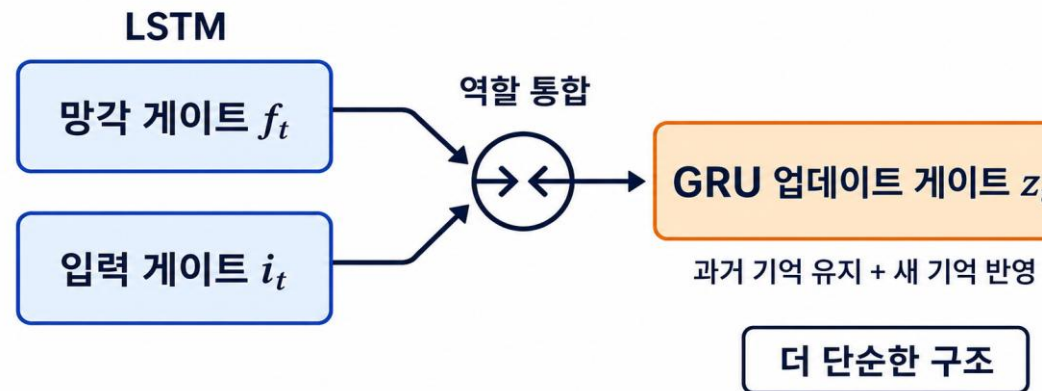
GRU의 의미 & LSTM과의 차이점

GRU의 의미

- ✓ LSTM 보다 단순화된 구조로, 망각 게이트와 입력 게이트를 하나로 합친 형태
- ✓ GRU는 출력 게이트가 존재하지 않아, 전체 상태 벡터가 매 단계마다 출력 되고, 이전 상태의 어느 부분이 출력이 될지 제어하는 새로운 게이트 컨트롤러가 존재한다.

LSTM과의 차이점

- ✓ LSTM: 망각·입력·출력 게이트, 셀 상태 C_t 은닉 상태 h_t
 - ✓ GRU: 업데이트·리셋 게이트, 은닉 상태 h_t 한개
- ⇒ 속도·단순성·적은 파라미터가 중요하면 GRU, 장기 기억을 세밀하게 제어하는 능력이 중요하면 LSTM을 우선 고려하되, 최종 선택은 동일 조건의 검증 성능으로 결정해야 한다



업데이트 게이트 & 리셋 게이트

업데이트 컨트롤러

- ✓ 의미
: 과거 기억과 새로운 후보 기억을 어떤 비율로 섞을지 결정

- ✓ 업데이트 게이트의 수식

$$z_t = \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z)$$
$$\Rightarrow h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

- ✓ 특징

- $z_t \approx 0$: 이전 기억 h_{t-1} 을 유지
- $z_t \approx 1$: 새로운 후보 기억 $\tilde{h}_t$ 으로 갱신

⇒ 업데이트 게이트는 LSTM의 망각 게이트와 입력 게이트 역할을 합친 것

추가적으로, z_t 의 역할을 반대로 표기해도 원리는 같다.

리셋 게이트

- ✓ 의미: r_t 는 새로운 후보 기억을 만들 때 과거 정보를 얼마나 참고할지를 결정

- ✓ 리셋 게이트의 수식

$$r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r)$$

- ✓ 특징

- $r_t \approx 0$: 과거 정보를 거의 무시
- $r_t \approx 1$: 과거 정보를 적극적으로 활용

후보군 & 최종 은닉 상태

✓ 후보군

: 리셋 게이트를 통과한 이전 은닉 상태와 현재 입력을 이용해 새 후보 기억을 만들.

✓ 후보군 수식

$$\tilde{h}_t = \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h)$$

[$\odot$ 의 의미는 "점 단위 연산". 즉, 각각의 차원에 맞게 곱하거나 더하는 부호]

추가적으로, $r_t \odot h_{t-1}$ 해당 부분이 작으면 이전 정보가 제거된 상태에서 새 기억을 계산한다.

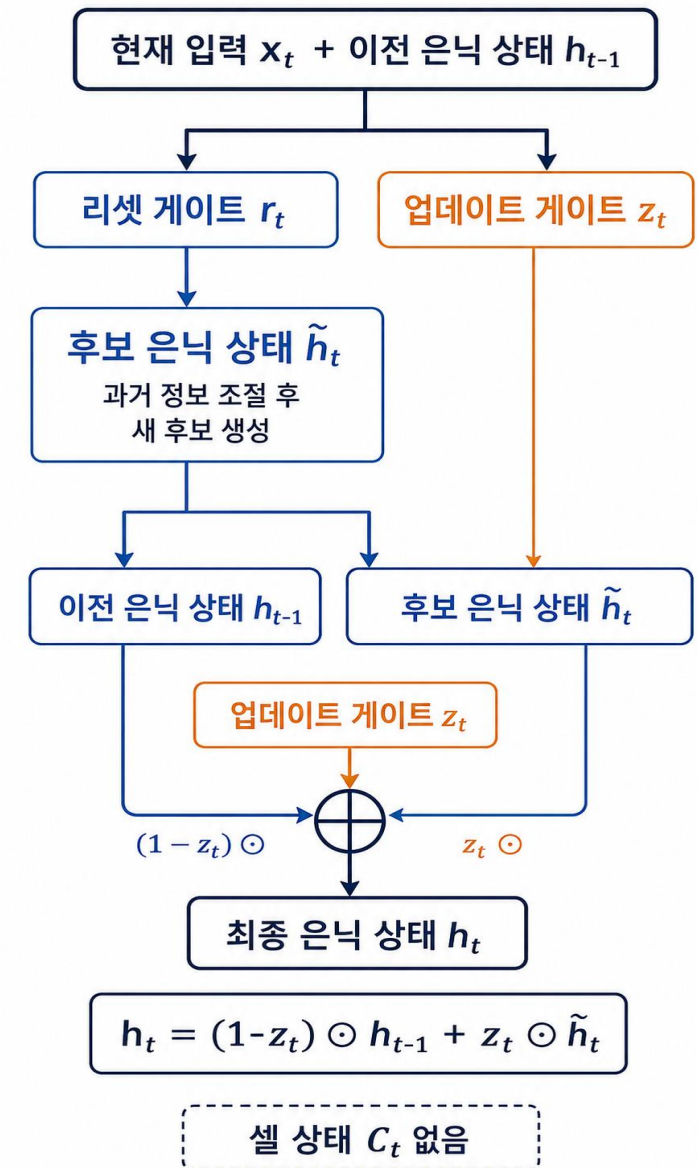
✓ 최종 은닉 상태

: h_t 는 이전 은닉 상태와 새로 만든 후보 은닉 상태를 업데이트 게이트로 합친 결과

✓ 최종 은닉 상태 수식 (현재 기억 = 유지한 과거 기억 + 선택한 새 기억)

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

최종 GRU 구조



■ 셀 구현 예제에서의 LSTM vs GRU

Part 1
Introduction

LSTM * 4?

✓ LSTM 셀 구현에서 왜 은닉층에 4를 곱하는가?

⇒ LSTM의 게이트는 망각, 입력, 셀, 출력 총 4가지로 구성되어 있다. 여기서 셀 구축 시 `gates.chunk(4, 1)`에 의해서 네 개로 쪼개지기 때문에, 최종적으로, 은닉층 뉴런/유닛에 4를 곱해줘야 한다

Part 2
Method

LSTM 셀 구현

```
class LSTMcell(nn.Module):  
    def __init__(self, input_size, hidden_size, bias=True):  
        super(LSTMcell, self).__init__()  
        self.input_size = input_size  
        self.hidden_size = hidden_size  
        self.bias = bias  
        self.x2h = nn.Linear(input_size, 4 * hidden_size, bias=bias)  
        self.h2h = nn.Linear(hidden_size, 4 * hidden_size, bias=bias)  
        self.reset_parameters()
```

```
gates = F.linear(input, w_ih, b_ih) + F.linear(hx, w_hh, b_hh)
```

```
def forward(self, x, hidden):  
    hx, cx = hidden  
    x = x.view(-1, x.size(1))
```

```
gates = self.x2h(x) + self.h2h(hx)
```

```
gates = gates.squeeze()
```

```
ingate, forgetgate, cellgate, outgate = gates.chunk(4, 1)
```

Part 3
Results &
Discussion

Part 4
Conclusion

■ 셀 구현 예제에서의 LSTM vs GRU

GRU * 3?

✓ GRU 셀 구현에서 왜 은닉층에 3을 곱하는가?

⇒ GRU의 게이트는 LSTM과 다르게 업데이트, 리셋 + 탄젠트 활성화 함수가 적용되는 부분까지 새로운 게이트로 정의 하여 총 3가지로 나누어지기 때문에, 최종적으로, 3을 곱해준다.

GRU 셀 구현

```
class GRUCell(nn.Module):
    def __init__(self, input_size, hidden_size, bias=True):
        super(GRUCell, self).__init__()

        self.input_size = input_size
        self.hidden_size = hidden_size
        self.bias = bias
        self.x2h = nn.Linear(input_size, 3 * hidden_size, bias=bias)
        self.h2h = nn.Linear(hidden_size, 3 * hidden_size, bias=bias)
        self.reset_parameters()
```

```
def forward(self, x, hidden):
    x = x.view(-1, x.size(1))
```

```
    gate_x = self.x2h(x)
    gate_h = self.h2h(hidden)
```

```
    gate_x = gate_x.squeeze()
    gate_h = gate_h.squeeze()
```

```
    i_r, i_i, i_n = gate_x.chunk(3, 1)
    h_r, h_i, h_n = gate_h.chunk(3, 1)
```

→ LSTM과 다른 이유는 GRU 셀에서는 개별적인 상태를 유지하기 때문

Any Questions?

이름 최준석

소속 한양대학교 기계공학과

이메일 chw10109@hanyang.ac.kr



*Intelligent Design
for Engineering
Applications Lab*

